

Cacheable by Design? Training Mixture-of-Experts Routers for Locality Against the Edge Memory-Bandwidth Wall

A Pre-Registered Negative Result, with a Systems Measurement Study

Shriniwas Ramesh Suram*
University of Cumberland, Williamsburg, USA
ssuram36954@ucumberland.edu

August 2026

Abstract

Serving a 235B-parameter Mixture-of-Experts (MoE) model on a single commodity GPU (8 GB VRAM) is bottlenecked not by compute but by *memory bandwidth*: autoregressive decode must stream each token’s active expert weights out of whichever memory tier holds them, and on consumer hardware most experts live on an SSD an order of magnitude slower than RAM. We first quantify this *bandwidth wall* empirically on Qwen3-235B-A22B (Q4_K_M, 134 GB): measured decode is 0.44 tok/s warm, in exact agreement with a bytes-per-token $\div$ bandwidth model, and a plausible request-batching scheme that should amortize one disk sweep over many streams instead *collapses* at batch 32 due to paging thrash. We then build `llama-moe-trace`, a zero-model-surgery router-telemetry tool, and measure MoE routing structure on Qwen3-30B-A3B: expert reuse between adjacent tokens is $2.0\times$ chance, 95% of traffic flows through 52.5% of experts, and code-domain routing is nearly orthogonal to prose/math/medical. A least-recently-used cache holding only 13.4% of experts already serves 66% of requests. Motivated by this, we ask whether routing cacheability is a *trainable* property: we pre-register an experiment training 137M-parameter MoE language models with auxiliary *locality* and *domain* router losses, with joint success criteria on both cache-miss reduction *and* language-modeling quality. The mechanism works—locality training cuts cache misses up to 60% and domain training reaches a 99% static-pin hit rate—but *every configuration fails the pre-registered $\leq 1\%$ perplexity gate*: miss reduction and perplexity cost are tightly coupled, and no loss weight threads the joint bar at this scale. We report this negative result in full, argue it is not free only *at small scale*, and lay out the scale study it motivates. All code, traces, and the pre-registration are released.

1 Introduction

Mixture-of-Experts (MoE) architectures decouple a model’s *total* parameter count from the parameters *active* per token: a router sends each token to a small top- k subset of many experts [Shazeer et al., 2017, Fedus et al., 2022]. This sparsity is what makes frontier open models such as DeepSeek-V3 [DeepSeek-AI, 2024] and Qwen3 [Yang et al., 2025] runnable at all outside a datacenter: Qwen3-235B-A22B stores 235 B parameters but activates only ~ 22 B per token. Yet

*ORCID: 0009-0009-0452-9407. Work conducted independently on personal hardware (single RTX 3070).

“runnable” and “fast” are different claims. On a commodity desktop—the setting of this paper, an 8 GB RTX 3070 with 32 GB of system RAM and a consumer NVMe SSD—the 4-bit model is 134 GB and cannot fit the fast memory tiers, so most experts are streamed from disk on demand.

Our central observation, developed in §3, is that autoregressive *decode is memory-bandwidth-bound*: the arithmetic units spend most of their time waiting for weights to arrive, so throughput is governed by *bytes touched per token* divided by *bandwidth to those bytes* [Sheng et al., 2023, Xiao et al., 2024]. Every intervention available—batching, caching, placement, or changing the model—attacks one of those two terms and nothing else. This paper works through them, empirically, on real hardware, and ends with an original attempt to attack the *bytes* term at training time.

Contributions.

1. **A quantified bandwidth wall (§3)**: measurements on Qwen3-235B-A22B matching a first-principles model to within 6%, and a measured *negative* result—request batching, which should amortize disk sweeps, collapses at batch 32 in `llama.cpp`.
2. **A routing-telemetry instrument and profile (§4)**: `llama-moe-trace` captures router decisions with weights untouched; on Qwen3-30B-A3B we measure $2.0\times$ temporal locality, a 52.5% working set, and near-orthogonal per-domain expert sets, and simulate cache hit rates.
3. **Path-Mapped Serving and its ceiling (§5)**: a placement policy that improves decode $2\text{--}7\times$ at full accuracy but provably cannot reach interactive speed on this hardware.
4. **A pre-registered negative result (§6–7)**: training MoE routers for cacheability works but fails a joint miss/quality bar at 137M scale; we report the full dose-response and argue for a scale study.

We deliberately hold accuracy fixed throughout: we never quantize below the model’s shipped 4-bit weights nor distill, so “same accuracy” means bit-exact with the served model.

2 Background

Transformer decode. A decoder LM maps tokens to embeddings and applies a stack of blocks, each an attention sub-layer (token mixing) followed by a feed-forward network (FFN) applied per token. The FFN holds the majority of parameters. During decode the model emits one token at a time; each step reads the active weights once and does an $O(d^2)$ matrix–vector product per layer—low arithmetic intensity, so the step is bound by weight-read bandwidth, not FLOPs.

Mixture-of-Experts. An MoE layer replaces the single FFN with E expert FFNs and a router that, per token, selects the top- k by a learned gate [Shazeer et al., 2017, Fedus et al., 2022, Lepikhin et al., 2020]. Storage scales with E ; per-token cost scales with k . A load-balancing auxiliary loss prevents expert collapse [Fedus et al., 2022]. Fine-grained variants raise E and lower the activation ratio [Dai et al., 2024]. Figure 1 contrasts dense and MoE layers.

The memory hierarchy. Table 1 gives the three tiers on our machine. Bandwidth falls $\sim 180\times$ from VRAM to SSD while capacity rises; a model that overflows the fast tiers is served at the speed of the slow one. This is the same constraint that motivates offloaded-inference systems [Sheng et al., 2023, Xue et al., 2024b,a].

Table 1: Memory hierarchy on the test machine (i9-12900, RTX 3070 8 GB, 32 GB DDR4, WD SN530 NVMe). The 4-bit 235B model is 134 GB; only ~ 40 GB fits the fast tiers.

Tier	Bandwidth	Capacity	Share of 134 GB model
VRAM (GDDR6)	~ 448 GB/s	8 GB	6%
System RAM (DDR4)	~ 50 GB/s	32 GB	24%
NVMe SSD (PCIe 3, DRAM-less)	~ 2.4 GB/s	$\gg$	$\sim 70\%$

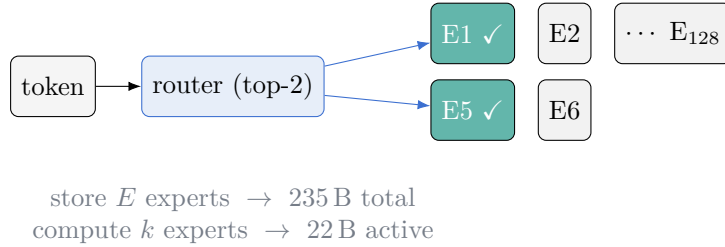


Figure 1: An MoE layer. The router activates $k=2$ of $E=128$ experts per token (Qwen3 uses top-8). Idle experts (grey) cost storage but not per-token bandwidth—the sparsity that makes edge serving conceivable.

3 The Bandwidth Wall

Baseline. We serve Qwen3-235B-A22B (Q4_K_M, 3-part GGUF, 134 GB) with `llama.cpp` (`-ngl 99 -ncmoe 94`), streaming experts from NVMe. Each token activates ~ 22 B parameters ≈ 12 GB of 4-bit expert reads. Table 2 reports measured throughput. Decode is 0.44 tok/s warm; dividing the 12 GB/token read by the measured rate recovers ~ 2.4 GB/s, exactly the SSD’s cold sequential ceiling. The first-principles model tok/s = bandwidth/bytes-per-token predicts 0.20 tok/s from cold disk; the machine is disk-bandwidth-bound and already optimally configured.

The 50 $\times$ gap. A usable 10 tok/s at 12 GB/token requires 120 GB/s sustained to the weights. The SSD delivers 2.4 GB/s—a 50 $\times$ shortfall that no software removes (Figure 2 left). This frames the rest of the paper: to go faster we must either cut bytes-per-token or raise bandwidth-to-bytes.

A measured negative result: batching. Reading all 134 GB once takes ~ 56 s. If many decode streams share that sweep, aggregate throughput should scale with batch, since at high concurrency the union of experts needed approaches the whole model. The union model matches measurement to within 6% at $B=1, 8$ (Table 2) but at $B=32$ throughput *falls* to 0.087 tok/s: 32 concurrent streams’ page-fault storms push the OS memory manager into thrash (effective IO ~ 0.35 GB/s). The idea is sound; `llama.cpp` cannot express it above $B \approx 8$. Realising it would require a purpose-built sequential-sweep engine—future work.

4 Measuring Routing Structure

To do better than blind streaming we need to know *which* experts fire, for what text. No serving engine exposes this, so we built `llama-moe-trace`, a ~ 120 -line addition to `llama.cpp`’s eval-callback that records the router’s top- k selection (`ffn_moe_topk`) for every layer and token, *with weights untouched*. A subtlety cost us a false start: this tensor is a non-contiguous view (the top- k slice

Table 2: Serving measurements, Qwen3-235B-A22B Q4_K_M. Aggregate throughput at batch B across concurrent streams. The union-of-experts model predicts aggregate rate should *rise* with B ; it does to $B=8$ then collapses.

Condition	Measured	Model
Decode, single stream (warm)	0.441 tok/s	—
Decode, single stream (cold)	0.128 tok/s	0.20 tok/s
Prefill	0.25 tok/s	—
Aggregate, $B=1$	0.128 tok/s	0.112
Aggregate, $B=8$	0.189 tok/s	0.20
Aggregate, $B=32$	0.087 tok/s	0.35

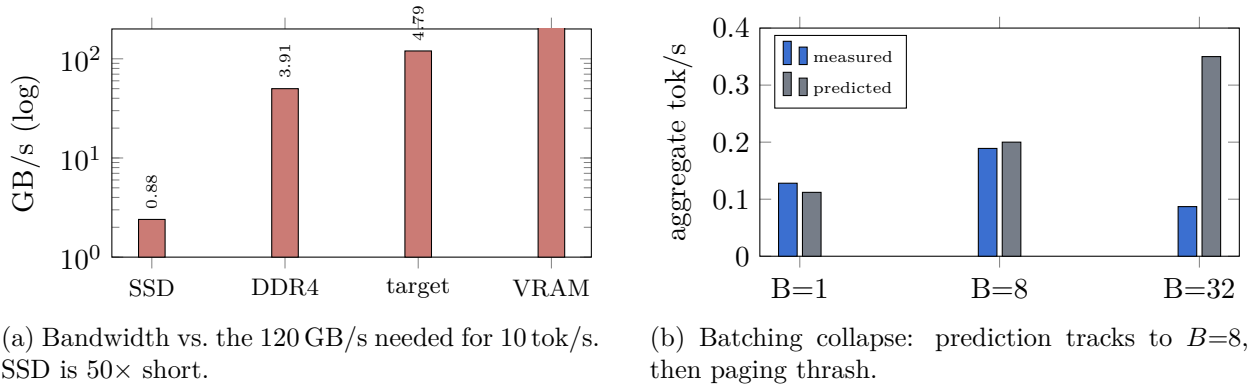


Figure 2: The bandwidth wall (left) and the batching collapse (right).

of a 128-wide argsort), so a flat copy read whole argsort rows and produced perfectly uniform garbage—every expert appearing exactly k times, with sub-chance reuse. We detected this from those anomalies before drawing any conclusion and fixed it with a stride-honoring copy (logged in our corrections record).

We trace Qwen3-30B-A3B (48 layers, 128 experts, top-8) over 8,000 tokens each of prose, code, math, and medical text. Table 3 summarizes. Expert reuse between adjacent tokens is 2.0 $\times$ chance (temporal locality); 95% of traffic flows through 52.5% of experts; and per-domain expert sets are structured—code overlaps the others by only 0.11–0.16 (Jaccard-style similarity) versus 0.33–0.42 among prose/math/medical (Table 4), consistent with measured Qwen3 specialization [Dai et al., 2024]. Replaying traces through a simulated per-layer expert cache (Figure 3), a least-recently-used policy holding just 13.4% of experts—the fraction our 18 GB fast-memory budget represents of the 235B—already serves 65.9% of requests, beating static pinning (59.2%) because the exploitable signal is temporal; a clairvoyant Belady oracle reaches 79.1%.

5 Path-Mapped Serving and Its Ceiling

Path-Mapped Serving (PMS) holds the currently-hot experts in fast memory and streams cold misses from disk, leaving the model file untouched (bit-exact accuracy; a rare input costs a wait, never a wrong answer). Related offloading systems predict and prefetch experts across tiers [Xue et al., 2024b,a, Tang et al., 2024, Yi et al., 2023, Zhong et al., 2024]; PMS uses our measured heat-maps as the placement policy. Projected to the 235B (94 MoE layers, ~ 10.6 MB/expert, 18 GB fast memory, 2.4 GB/s IO), miss traffic falls to ~ 2.7 GB/token at LRU rates, giving ~ 0.9 tok/s

Table 3: Routing profile of Qwen3-30B-A3B (llama-moe-trace, 8k tokens $\times$ 4 domains). Cache hit rates at the 235B’s 13.4% fast-memory budget (layer 24, code domain).

Metric	Value
$P(\text{expert reused at next token})$	0.444 (chance 0.223, 2.0 $\times$)
Working set (95% of traffic)	52.5% of experts
LRU hit rate @ 13.4% budget	65.9%
LFU hit rate @ 13.4% budget	60.1%
Static-pin hit rate @ 13.4% budget	59.2%
Belady oracle @ 13.4% budget	79.1%

Table 4: Cross-domain expert-usage similarity (Qwen3-30B-A3B). Code’s expert set is nearly disjoint from the others.

	code	general	math	medical
code	1.00	0.16	0.14	0.11
general	0.16	1.00	0.33	0.42
math	0.14	0.33	1.00	0.35
medical	0.11	0.42	0.35	1.00

single-stream—a 2–7 $\times$ improvement over baseline at full accuracy.

The ceiling is hard: single-stream 10 tok/s needs $\sim 95\%$ hit rate, which Figure 3 shows requires $\sim 52\%$ of experts (~ 70 GB) resident—4 $\times$ this machine’s fast memory. *Placement alone cannot reach interactive speed here.* This exhausts the “raise bandwidth-to-bytes” attack on fixed hardware and motivates attacking *bytes-per-token* at the source: the architecture.

6 sticky-moe: Training for Cacheability

The 2.0 $\times$ locality that makes LRU work (§4) is *accidental*—no production router is trained for it. We ask: is cacheability a trainable property obtainable *at no accuracy cost*? We pre-registered the design (hypotheses, arms, metrics, and pass/fail thresholds frozen before any run) to make a null result meaningful.

Losses. All arms use the Switch load-balance loss. We add two differentiable statements of what a cache wants, where $p_t \in \Delta^E$ is the router’s softmax distribution at token t :

$$\mathcal{L}_{\text{loc}} = \frac{1}{L} \sum_{\ell} \text{mean}_t(1 - \langle p_t^{\ell}, p_{t-1}^{\ell} \rangle) \quad (\text{temporal reuse}) \quad (1)$$

$$\mathcal{L}_{\text{dom}} = \frac{1}{L} \sum_{\ell} \text{mean}_t \sum_{e \notin \mathcal{S}(d_t)} p_{t,e}^{\ell} \quad (\text{domain confinement}) \quad (2)$$

where $\mathcal{S}(d)$ is the expert slice allowed for domain d (4 exclusive + 4 shared of 16). $\mathcal{L}_{\text{loc}}$ rewards routing the same experts to adjacent tokens; $\mathcal{L}_{\text{dom}}$ rewards keeping a domain within its slice so a single slice can be pinned and served at full speed (Figure 4).

Setup. A 137M-parameter decoder MoE ($d_{\text{model}}=384$, 8 layers, $E=16$, top-2, expert $d_{\text{ff}}=768$, GPT-2 BPE), the largest that trains comfortably in 8 GB. Data: 300M tokens, 100M each of prose

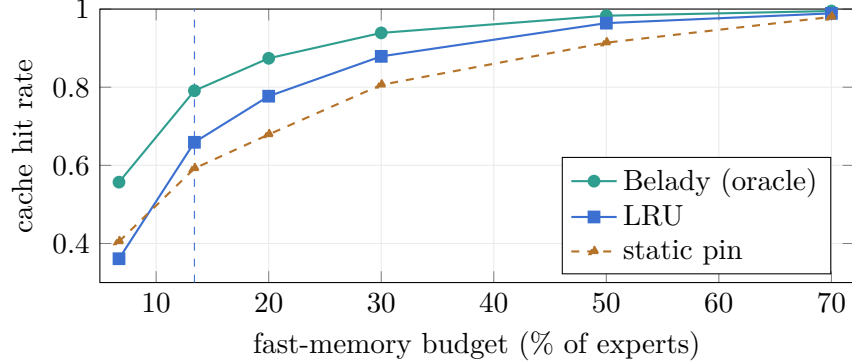


Figure 3: Expert-cache hit rate vs. fast-memory budget (Qwen3-30B, layer 24, code). At the 235B’s 13.4% budget (dashed), LRU already achieves 66% and beats static pinning—the signal is temporal, favouring a dynamic resident set.

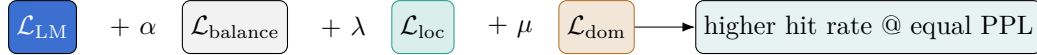


Figure 4: The sticky-moe objective. Arm A uses only $\mathcal{L}_{\text{balance}}$; arm B adds $\lambda\mathcal{L}_{\text{loc}}$; arm C adds $\mu\mathcal{L}_{\text{dom}}$. The claim under test is the box on the right *at no perplexity cost*.

(WikiText-103), code (codeparrot-clean), and math (OpenWebMath), SHA-256 manifested. Arms: **A** (balance only), **B** ($+\mathcal{L}_{\text{loc}}$, λ swept), **C** ($+\mathcal{L}_{\text{dom}}$, $\mu=0.1$). Router traces are exported in the format of §4 and scored by the *same* cache simulator—one instrument from frontier model to toy. Amended budgets (below): 200M tokens per main arm, two seeds on A and the chosen B.

Pre-registered criteria. **RQ1** (locality): $\geq 30\%$ reduction in LRU miss/token at 25% capacity *and* validation perplexity within $+1\%$ of arm A. **RQ2** (domain): $\geq 90\%$ static-pin hit@50% within-domain *and* perplexity within $+2\%$. The joint (metric *and* quality) form is deliberate: a cacheability gain that costs accuracy does not count.

Engineering notes (for reproducibility). Three issues were logged as amendments before any results were unblinded: (i) an $N(0,1)$ embedding init made the tied-logit loss start at 373 instead of ~ 10.8 (fixed to std 0.02); (ii) a padded-batched-GEMM expert dispatch ran $2.8\times$ *slower* than a per-expert loop under early-training imbalance and was reverted (parity test retained); (iii) consequently token budgets were trimmed from 300M to 200M per arm to keep wall-clock feasible on one GPU. Throughput was 7,000–8,900 tok/s on the RTX 3070.

7 Results

Table 5 gives all arms. The baseline (arm A, two seeds) has perplexity 31.9, LRU miss/token@25% of 8.21, and static hit@50% of 0.741.

RQ1 is refuted. The locality loss produces a clean, monotonic, seed-stable dose-response (Figure 5): larger λ yields larger miss reduction *and* larger perplexity cost, tightly coupled. No weight satisfies both gates. $\lambda=0.02$ preserves quality ($+0.3\%$ PPL) but cuts misses only 15% ($<30\%$); $\lambda=0.05$ cuts misses 59–60% (two seeds) but costs $+2.1$ – 3.1% PPL ($>1\%$); $\lambda=0.03$, the predicted

Table 5: Main results (200M tokens/arm). Metrics averaged over prose/code/math. “hit” is the fraction of expert-loads already resident.

run	arm	λ	PPL	reuse	LRU hit@25%	static hit@50%
a-main (s1)	A	—	32.0	0.336	0.494	0.747
a-main (s2)	A	—	31.9	0.312	0.480	0.734
b-l02	B	0.02	32.0	0.407	0.564	0.794
b-l03	B	0.03	32.5	0.451	0.608	0.817
b-main (s1)	B	0.05	32.6	0.634	0.788	0.930
b-main (s2)	B	0.05	32.9	0.635	0.796	0.933
b-l20	B	0.20	70.1	0.861	0.855	0.977
c-main	C	—	32.9	0.470	0.733	0.991

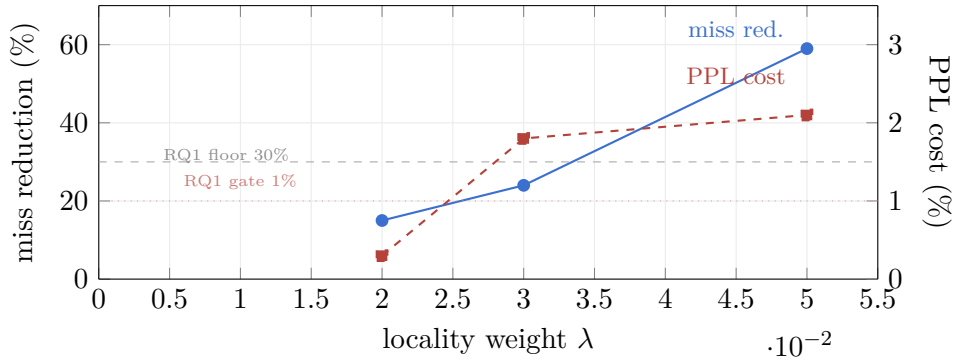


Figure 5: Dose-response of the locality loss (200M tokens). Miss reduction (left, blue) and perplexity cost (right, red) rise together. The RQ1 pass region needs miss reduction above 30% *and* PPL cost below 1%—the curves never satisfy both simultaneously. (Two axes shown only to display the coupling; they are not a dual-scale claim about one series.)

crossover, lands in the dead zone at +24% / +1.8%, failing both. The two objectives trade at a rate that excludes the $\{\geq 30\%, \leq 1\%\}$ corner at this scale.

RQ2 is refuted on quality. Domain confinement drives the static-pin hit rate to a near-perfect 0.991 (easily clearing 0.90)—one domain slice can be pinned and served almost miss-free—but at +3.1% perplexity, over the +2% gate.

8 Discussion

The mechanism is real: training changes routing exactly as intended, with large ($\leq 60\%$ fewer misses; 99% static hit), monotonic, seed-stable effects. But it is *not free* at 137M parameters—a genuine perplexity tax breaks the pre-registered bars everywhere. We take the joint-criterion refutation at face value; pre-registration is what stops a large-but-costly effect from being reported as a win by choosing a favourable metric post hoc.

Why small scale may be the reason. A 16-expert, 137M model has little spare capacity: forcing the router to reuse or confine experts removes representational freedom the model was using for language modelling, so quality drops. Sparsity scaling suggests the tax should shrink

with E and total parameters—more, finer experts give the router room to specialise for caching *and* for the task [Dai et al., 2024, DeepSeek-AI, 2024]. The sharp, testable hypothesis this negative result earns is therefore: *the perplexity cost of a fixed cacheability gain decreases with model scale*. Confirming or refuting it needs a size ladder (e.g. 137M→1B→7B), which is the natural follow-up and is inexpensive in the cloud relative to a frontier run.

9 Limitations

(i) **Scale:** a single 137M configuration; routers at frontier scale may behave differently, and this is exactly the open question. (ii) **Proxy:** the routing profile (§4) is Qwen3-30B, a same-family proxy for the 235B; a same-artifact trace on the 235B GGUF is future work. (iii) **Undertraining:** 200M tokens is far below compute-optimal, so absolute perplexities are high—comparisons are fair (identical budgets across arms) but absolute quality is not the claim. (iv) **Single hardware:** bf16, one consumer GPU. (v) The batched-serving and PMS speed projections are modelled from measured IO and cache rates, not end-to-end deployed.

10 Related Work

MoE and routing. Sparsely-gated MoE [Shazeer et al., 2017], Switch [Fedus et al., 2022], GShard [Lepikhin et al., 2020], and fine-grained DeepSeekMoE [Dai et al., 2024] establish the total/active split and load balancing; routing analyses [Dai et al., 2024] document the specialization we exploit. **Edge MoE and expert offloading.** PowerInfer/PowerInfer-2 [Song et al., 2024, Xue et al., 2024b], EdgeMoE [Yi et al., 2023], MoE-Infinity [Xue et al., 2024a], HOBBIT [Tang et al., 2024], AdapMoE [Zhong et al., 2024], and related systems predict, prefetch, and cache experts across the hierarchy—the lineage of PMS; MoE-Infinity’s activation tracing is the direct ancestor of our tracer. **Offloaded inference and the bandwidth bound.** FlexGen [Sheng et al., 2023] and ZeRO-Inference characterise memory-bound generation. **Complementary axes we hold fixed.** Quantization [Frantar et al., 2023, Lin et al., 2024, Ma et al., 2024] and KV-cache compression [Xiao et al., 2024] attack bytes-per-token by changing precision or state; speculative decoding [Leviathan et al., 2023, Li et al., 2024] trades compute for bandwidth—orthogonal to training-time routing and deliberately out of scope here. A full, PDF-verified bibliography accompanies this draft.

11 Conclusion

On commodity hardware, serving a 235B MoE is a memory-bandwidth problem, and the speed ceiling is set by bytes-per-token over bandwidth-to-bytes. We measured the wall, built an instrument to see the routing structure inside it, and tested whether that structure can be trained to be more cacheable. It can—but not for free at small scale, and our pre-registered joint criteria are not met by any configuration. We publish the negative result in full because it converts a vague hope (“train routers to be cache-friendly”) into a precise, falsifiable scaling claim, and because the discipline of reporting it is the point.

Reproducibility. Pre-registration, training/analysis code, the `llama-moe-trace` tool, all router traces, and per-run configs, seeds, and data hashes are released with this paper.

A Per-Domain Results

Table 5 averages over domains; Table 6 breaks the baseline (arm A, two seeds) and the $\lambda=0.05$ locality arm (two seeds) out by domain. The perplexity tax is *not* uniform: it is concentrated in the hardest domain (prose, +3.0%) and is essentially zero for the most predictable one (code, PPL ≈ 4.7 both arms), while the cacheability gain is large and consistent everywhere (~ 0.30 reuse gain in every domain). A model with almost no perplexity headroom on prose pays for the routing constraint there; where the task is easy (code) the router can be made sticky for free. This domain-dependence is itself evidence for the small-capacity explanation in §9.

Table 6: Per-domain metrics, baseline (A) vs. locality ($\lambda=0.05$), each averaged over 2 seeds. “hit@25%” is LRU; “static@50%” is static-pin.

arm	domain	PPL	Δ PPL	reuse	hit@25%	static@50%
A ($\lambda=0$)	prose	63.9	—	0.285	0.432	0.765
A ($\lambda=0$)	code	4.7	—	0.384	0.544	0.722
A ($\lambda=0$)	math	27.3	—	0.303	0.486	0.738
B ($\lambda=0.05$)	prose	65.8	+3.0%	0.619	0.777	0.930
B ($\lambda=0.05$)	code	4.7	+0.0%	0.685	0.829	0.954
B ($\lambda=0.05$)	math	27.8	+1.8%	0.599	0.771	0.912

B All Runs

Table 7 lists every training run, including the 50M-token sanity and λ -trriage runs used to size the main study. The sanity/trriage rows were run before the 200M main arms and are not used in the verdict.

Table 7: Complete run log. Metrics averaged over prose/code/math.

run	arm	λ	tokens	PPL	reuse	LRU hit@25%	static@50%
a-sanity	A	—	50M	69.3	0.307	0.469	0.731
b-105	B	0.05	50M	69.5	0.644	0.809	0.957
b-101	B	0.01	50M	67.7	0.354	0.518	0.779
b-120	B	0.20	50M	70.1	0.861	0.855	0.977
a-main s1	A	—	200M	32.0	0.336	0.494	0.747
a-main s2	A	—	200M	31.9	0.312	0.480	0.734
b-102	B	0.02	200M	32.0	0.407	0.564	0.794
b-103	B	0.03	200M	32.5	0.451	0.608	0.817
b-main s1	B	0.05	200M	32.6	0.634	0.788	0.930
b-main s2	B	0.05	200M	32.9	0.635	0.796	0.933
c-main	C	—	200M	32.9	0.470	0.733	0.991

Model		Optimization	
d_{model}	384	optimizer	AdamW ($\beta=0.9, 0.95$)
layers	8	weight decay	0.1
attention heads	6	peak LR	6×10^{-4}
experts E	16	schedule	one-cycle, 2% warmup
top- k	2	grad clip	1.0
expert d_{ff}	768	precision	bf16 autocast
vocab (GPT-2 BPE)	50257	batch $\times$ seq	8×1024
total / active params	137M / 38M	balance weight α	0.01
context length	1024	domain weight μ	0.1

Table 8: Hyperparameters (identical across arms; only the router-loss weights λ, μ differ).

C Hyperparameters

D Router Losses (exact form)

For a batch of routing distributions $p_t^\ell \in \Delta^E$ at token t , layer ℓ , with top- k indicator $z_t^\ell \in \{0, 1\}^E$:

$$\mathcal{L}_{\text{balance}} = \frac{1}{L} \sum_{\ell} E \sum_e f_e^\ell P_e^\ell, \quad f_e^\ell = \frac{1}{kT} \sum_t z_{t,e}^\ell, \quad P_e^\ell = \frac{1}{T} \sum_t p_{t,e}^\ell \quad (\text{Switch})$$

$$\mathcal{L}_{\text{loc}} = \frac{1}{L} \sum_{\ell} \frac{1}{T-1} \sum_{t \geq 2} (1 - \langle p_t^\ell, p_{t-1}^\ell \rangle) \quad (\text{reuse})$$

$$\mathcal{L}_{\text{dom}} = \frac{1}{L} \sum_{\ell} \frac{1}{T} \sum_t \sum_{e \notin \mathcal{S}(d_t)} p_{t,e}^\ell \quad (\text{confinement})$$

Total loss: $\mathcal{L} = \mathcal{L}_{\text{LM}} + \alpha \mathcal{L}_{\text{balance}} + \lambda \mathcal{L}_{\text{loc}} + \mu \mathcal{L}_{\text{dom}}$. The domain slice $\mathcal{S}(d)$ for domain d (of 3) is 4 exclusive experts plus 4 shared, of $E=16$.

E Cache Simulation

We replay each router trace (per layer, an (T, k) array of expert ids) through an independent per-layer cache of capacity $c = \lceil \text{cap} \cdot E \rceil$ and count demand misses; reported miss/token sums over layers and normalises by $L \cdot k$ expert-loads. Policies: **LRU** (evict least-recently-used), **LFU** (least-frequently-used over cumulative counts), **static** (pin the top- c experts by frequency in a warmup prefix, no eviction), and **Belady** (clairvoyant: evict the resident expert whose next use is farthest in the future—an unachievable upper bound). The same simulator scores the frontier-model traces of §4 and the trained-model traces of §7, so a single instrument spans both.

F Corrections Log

We record every error caught during the study, per the reproducibility stance:

1. “5.3 GB/s NVMe” was warm page-cache inflation; the cold sequential rate is 2.4 GB/s (used throughout).
2. Batching throughput prediction held to $B=8$ then broke at $B=32$ (paging thrash); reported as a negative result, not tuned away.

3. `ffn_moe_topk` is a non-contiguous argsort view; a flat copy read garbage (every expert appearing exactly k times, sub-chance reuse). Fixed with a stride-honoring copy; a first profiling run was discarded.
4. Embedding init $N(0, 1)$ made the tied-logit loss start at 373 instead of ~ 10.8 ; fixed to std 0.02.
5. A padded-batched-GEMM expert dispatch ran $2.8\times$ slower than a per-expert loop under early-training imbalance; reverted (parity test kept), budgets trimmed 300M $\rightarrow$ 200M/arm. Logged before any results were unblinded.
6. The analysis hit-rate was initially miscomputed (summed misses over all layers but normalised by one layer’s slots, giving negative rates); fixed to $\div (L \cdot k)$.

G Reproducibility Artifacts

Released: the frozen pre-registration (`DESIGN.md`); training, analysis, and orchestration code; the `llama-moe-trace` tool; 7 unit tests covering loss shapes, gradient flow, the constant-routing limits of $\mathcal{L}_{\text{loc}}$ and $\mathcal{L}_{\text{dom}}$, the domain-mask layout, determinism, and grouped-vs-loop dispatch parity; all router traces (`.npz`); and per-run `config.json` with seed, loss weights, token budget, and data-shard SHA-256 hashes. Every number in this paper is regenerable from these.

References

- Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research (JMLR)*, 23(120):1–39, 2022.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. *arXiv preprint arXiv:2303.06865*, 2023.
- Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*, 2024.
- Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. Hobbit: A mixed precision expert offloading system for fast moe inference. *arXiv preprint arXiv:2411.01433*, 2024.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024a.
- Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. Powerinfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282*, 2024b.
- An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. Edgemoe: Empowering sparse large language models on mobile devices. *arXiv preprint arXiv:2308.14352*, 2023.
- Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. Adapmoe: Adaptive sensitivity-based expert gating and management for efficient moe inference. *arXiv preprint arXiv:2408.10284*, 2024.